

QUERY PROCESSING LAB EXP 20 to 30

NAME : GUNA P

REG NO : 192424277

COURSE CODE : DSA0503 - Query Processing

EXP 31

Aim

To write a Python program to create a stacked bar plot with error bars. The women bars are stacked on top of the men bars.

Algorithm

- Import NumPy and Matplotlib.
- Define groups.
- Define men's mean values.
- Define women's mean values.
- Define standard deviations.
- Plot men's bars with error bars.
- Use bottom=men to stack women's bars.
- Add labels and legend.
- Display the graph.

Code

```
import numpy as np
import matplotlib.pyplot as plt

groups = ['Group 1', 'Group 2', 'Group 3', 'Group 4', 'Group 5']

men = [22, 30, 35, 35, 26]
women = [25, 32, 30, 35, 29]

men_sd = [4, 3, 4, 1, 5]
women_sd = [3, 5, 2, 3, 3]

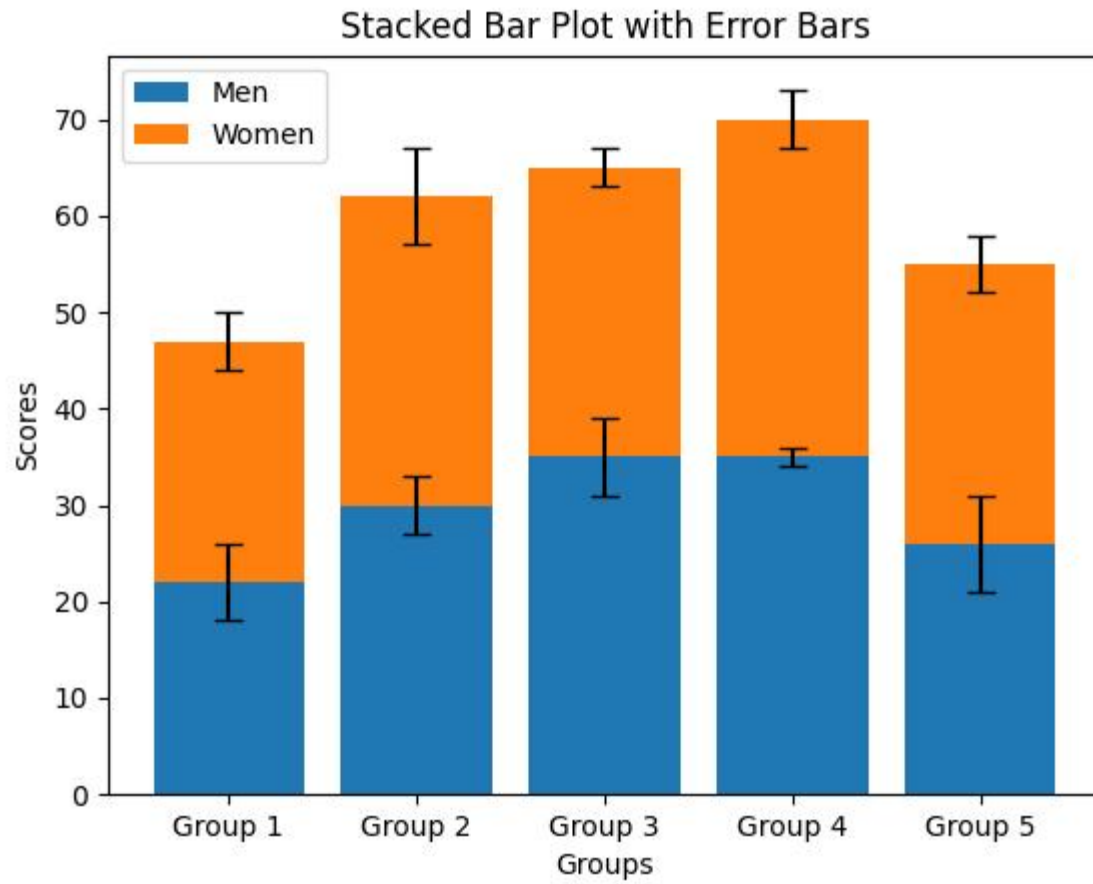
x = np.arange(len(groups))

plt.bar(x, men, yerr=men_sd, capsize=5, label='Men')
plt.bar(x, women, bottom=men, yerr=women_sd, capsize=5,
label='Women')

plt.xlabel("Groups")
plt.ylabel("Scores")
plt.title("Stacked Bar Plot with Error Bars")
plt.xticks(x, groups)
plt.legend()

plt.show()
```

Output



EXP 32

Aim

To write a Python program to draw a scatter graph using random distributions in X and Y.

Algorithm

- Import NumPy and Matplotlib.
- Generate random X values.
- Generate random Y values.
- Create a scatter plot.
- Add X-axis and Y-axis labels.
- Add title.
- Display the graph.

Code

```
import numpy as np
import matplotlib.pyplot as plt

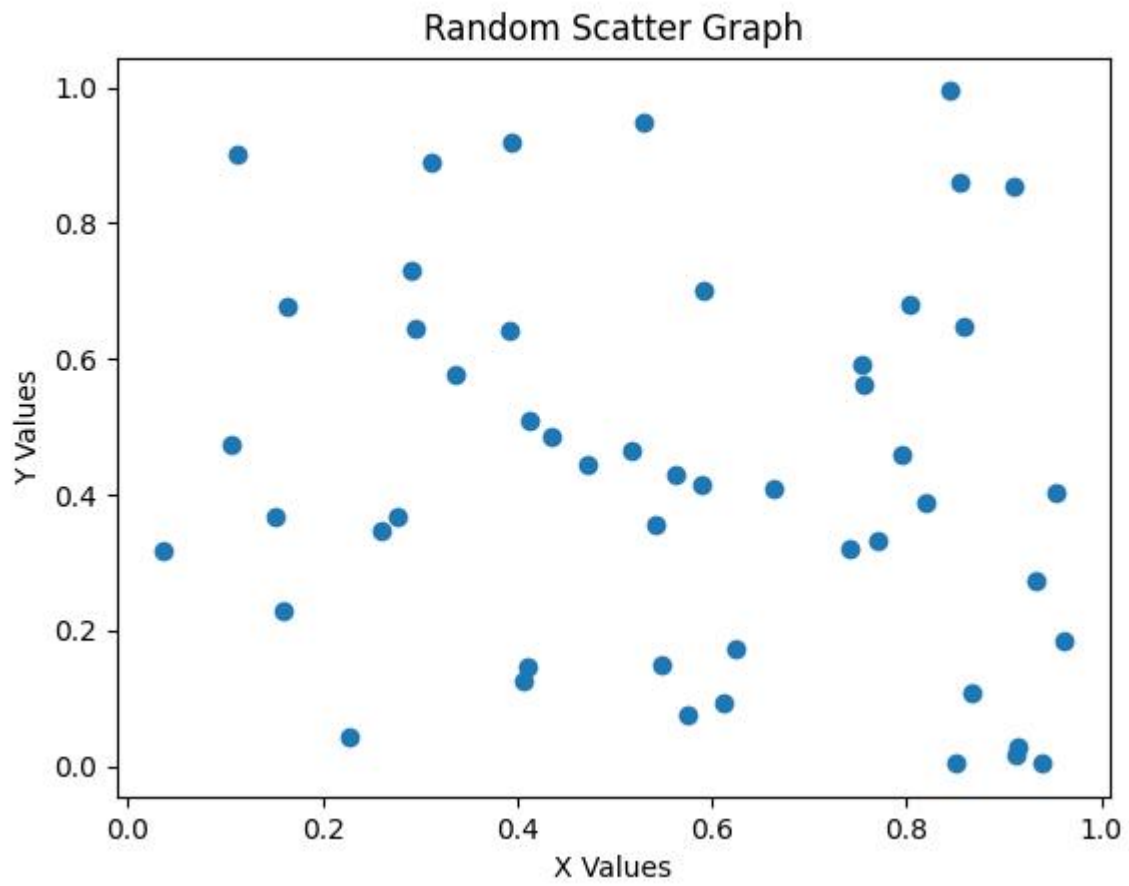
x = np.random.rand(50)
y = np.random.rand(50)

plt.scatter(x, y)

plt.xlabel("X Values")
plt.ylabel("Y Values")
plt.title("Random Scatter Graph")

plt.show()
```

Output



EXP 33

Aim

To write a Python program to draw a scatter plot with empty circles using random X and Y distributions.

Algorithm

- Import NumPy and Matplotlib.
- Generate random X values.
- Generate random Y values.
- Use scatter().
- Set the marker to o.
- Set the face color to none.
- Add labels and title.
- Display the graph.

Code

```
import numpy as np
import matplotlib.pyplot as plt

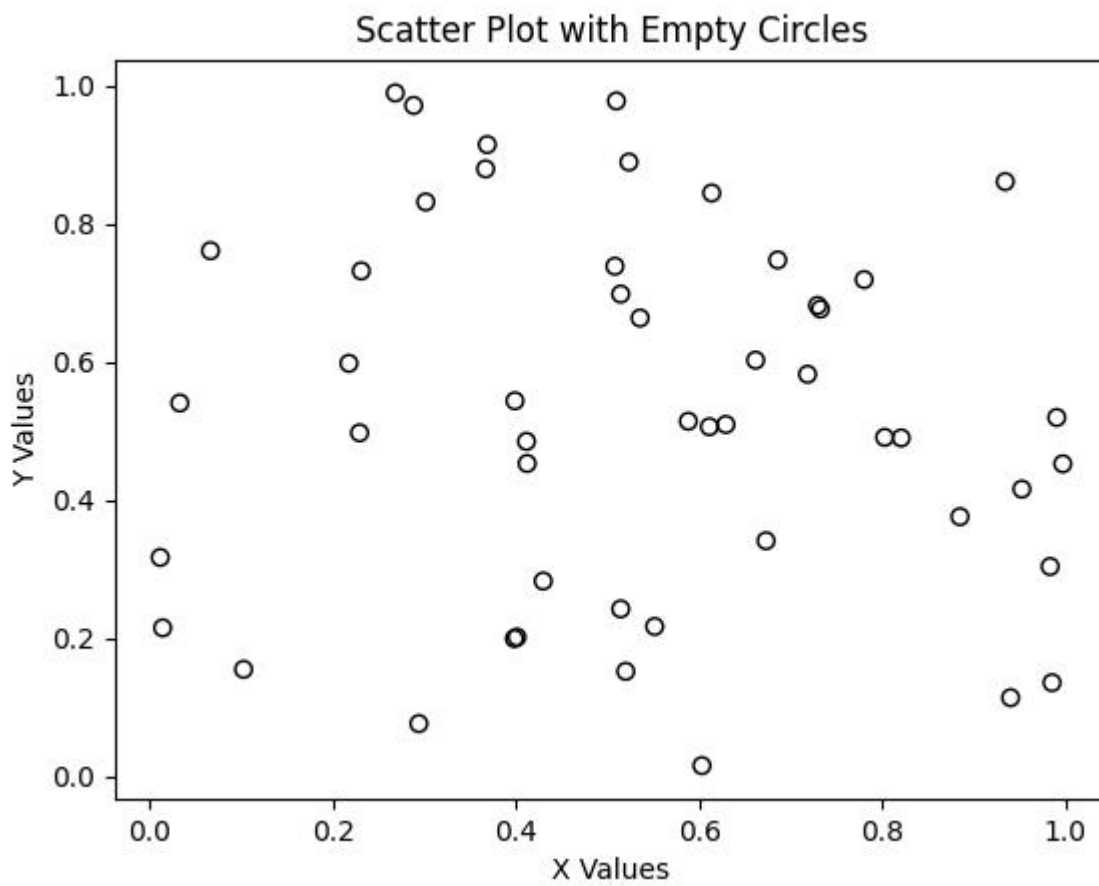
x = np.random.rand(50)
y = np.random.rand(50)

plt.scatter(x, y, marker='o', facecolors='none', edgecolors='black')

plt.xlabel("X Values")
plt.ylabel("Y Values")
plt.title("Scatter Plot with Empty Circles")
```

plt.show()

Output



EXP 34

Aim

To write a Python program to draw a scatter plot using random distributions to generate balls of different sizes.

Algorithm

- Import NumPy and Matplotlib.
- Generate random X values.
- Generate random Y values.
- Generate random sizes.
- Pass the sizes to the s parameter.
- Create the scatter plot.
- Add labels and title.
- Display the graph.

Code

```
import numpy as np
import matplotlib.pyplot as plt

x = np.random.rand(50)
y = np.random.rand(50)

sizes = np.random.randint(20, 500, 50)

plt.scatter(x, y, s=sizes)

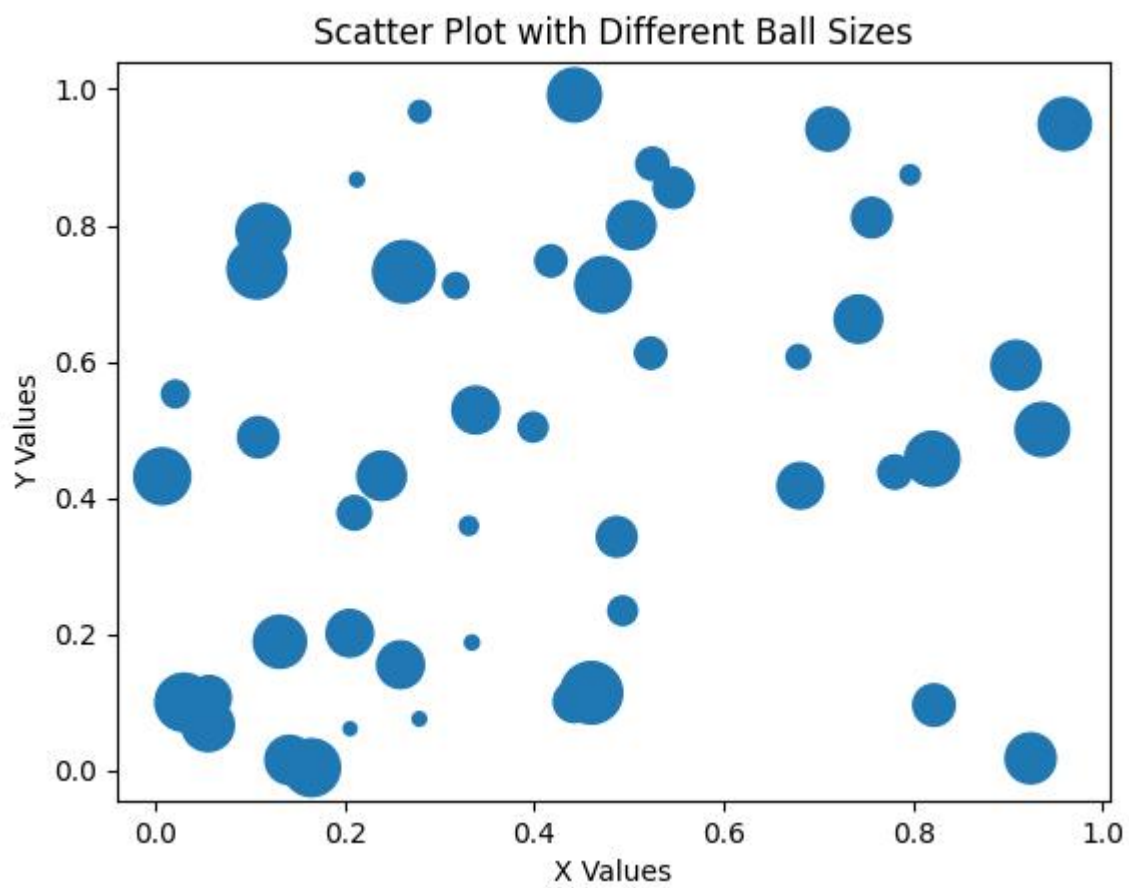
plt.xlabel("X Values")
plt.ylabel("Y Values")
```



```
plt.title("Scatter Plot with Different Ball Sizes")
```

```
plt.show()
```

Output



EXP 35

Aim

To write a Python program to draw a scatter plot comparing Mathematics and Science marks of 10 students. The experiment provides the Mathematics and Science mark values.

Algorithm

- Import Matplotlib.
- Store Mathematics marks.
- Store Science marks.
- Plot Mathematics marks against Science marks.
- Add X-axis label.
- Add Y-axis label.
- Add title.
- Display the scatter plot.

Code

```
import matplotlib.pyplot as plt
```

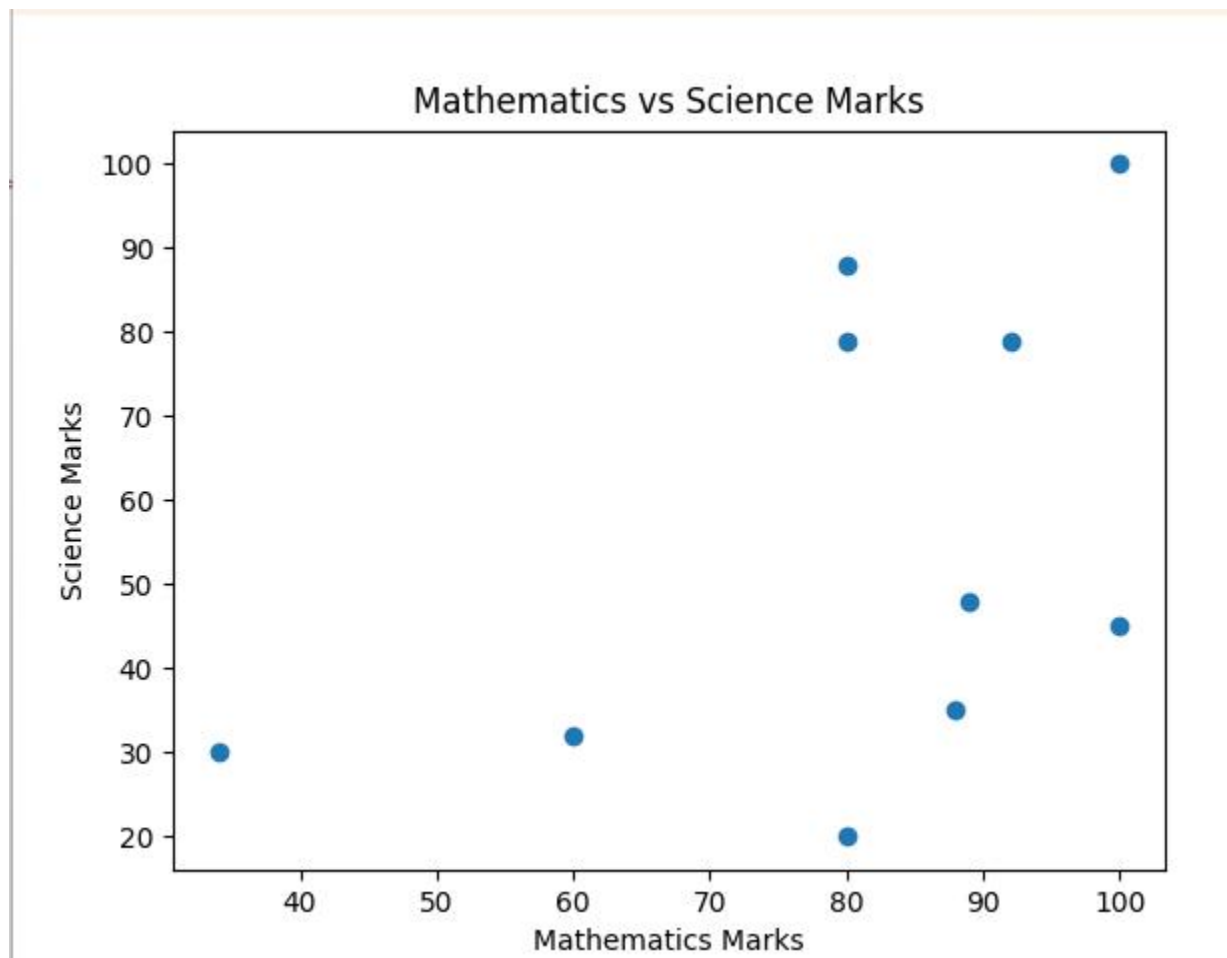
```
math_marks = [88, 92, 80, 89, 100, 80, 60, 100, 80, 34]  
science_marks = [35, 79, 79, 48, 100, 88, 32, 45, 20, 30]
```

```
plt.scatter(math_marks, science_marks)
```

```
plt.xlabel("Mathematics Marks")  
plt.ylabel("Science Marks")  
plt.title("Mathematics vs Science Marks")
```

```
plt.show()
```

Output



EXP 36

Aim

To write a Python program to draw a scatter plot for three different groups comparing weights and heights.

Algorithm

- Import Matplotlib.
- Define height and weight values for three groups.
- Plot Group 1.
- Plot Group 2.
- Plot Group 3.
- Add labels.
- Add legend.
- Display the scatter plot.

Code

```
import matplotlib.pyplot as plt
```

```
height1 = [150, 155, 160, 165, 170]
```

```
weight1 = [50, 53, 56, 60, 65]
```

```
height2 = [152, 158, 163, 168, 173]
```

```
weight2 = [52, 55, 59, 63, 68]
```

```
height3 = [148, 154, 161, 167, 175]
```

```
weight3 = [48, 52, 57, 62, 70]
```

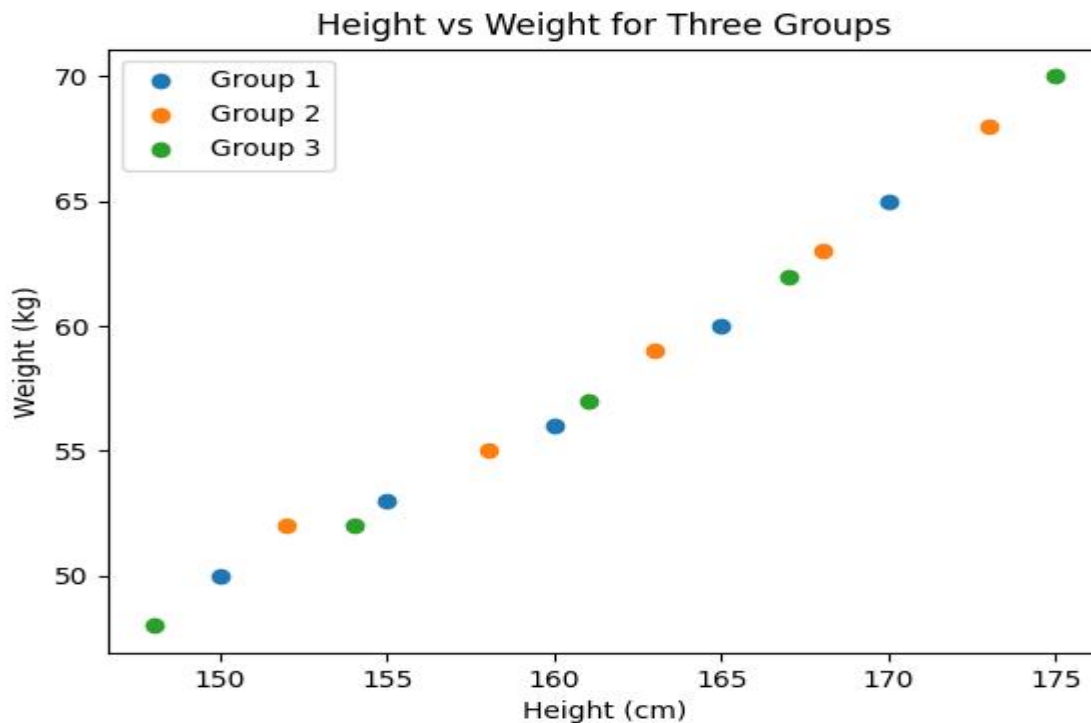
```
plt.scatter(height1, weight1, label='Group 1')
```

```
plt.scatter(height2, weight2, label='Group 2')
plt.scatter(height3, weight3, label='Group 3')

plt.xlabel("Height (cm)")
plt.ylabel("Weight (kg)")
plt.title("Height vs Weight for Three Groups")

plt.legend()
plt.show()
```

Output



EXP 37

Aim

To write a Pandas program to create a DataFrame from a dictionary and display it. The supplied dictionary contains columns X, Y and Z.

Algorithm

- Import Pandas.
- Create a dictionary.
- Pass the dictionary to `pd.DataFrame()`.
- Store it in a DataFrame.
- Display the DataFrame.

Code

```
import pandas as pd

data = {
    'X': [78, 85, 96, 80, 86],
    'Y': [84, 94, 89, 83, 86],
    'Z': [86, 97, 96, 72, 83]
}

df = pd.DataFrame(data)

print(df)
```

Output

```
===== RESTART: C:/Users/udhay/AppData/Local/Programs/Python/Python311/m.py =====
```

```
      X   Y   Z
0  78  84  86
1  85  94  97
2  96  89  96
3  80  83  72
4  86  86  83
```

EXP 38

Aim

To write a Pandas program to create and display a DataFrame from dictionary data with specified index labels. The experiment provides exam_data and labels a through j.

Algorithm

- Import Pandas and NumPy.
- Create the dictionary.
- Create the index labels.
- Pass the dictionary and labels to pd.DataFrame().
- Display the DataFrame.

Code

```
import pandas as pd
import numpy as np

exam_data = {
    'name': ['Anastasia', 'Dima', 'Katherine', 'James', 'Emily',
            'Michael', 'Matthew', 'Laura', 'Kevin', 'Jonas'],
    'score': [12.5, 9, 16.5, np.nan, 9, 20, 14.5, np.nan, 8, 19],
    'attempts': [1, 3, 2, 3, 2, 3, 1, 1, 2, 1],
    'qualify': ['yes', 'no', 'yes', 'no', 'no',
```



```
        'yes', 'yes', 'no', 'no', 'yes']
    }

labels = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j']

df = pd.DataFrame(exam_data, index=labels)

print(df)
```

Output

```
>>>
===== RESTART: C:/Users/udhay/AppData/Local/Programs/Python/Python311/m.py =====
      name  score  attempts  qualify
a  Anastasia   12.5         1     yes
b      Dima    9.0         3      no
c  Katherine   16.5         2     yes
d      James    NaN         3      no
e      Emily    9.0         2      no
f   Michael   20.0         3     yes
g   Matthew   14.5         1     yes
h      Laura    NaN         1      no
i      Kevin    8.0         2      no
j      Jonas   19.0         1     yes
>>>
```

EXP 39

Aim

To write a Pandas program to get the first three rows of a given DataFrame.

Algorithm

- Import Pandas and NumPy.
- Create the given DataFrame.
- Use the head(3) function.
- Display the first three rows.

Code

```
import pandas as pd  
import numpy as np
```

```

exam_data = {
    'name': ['Anastasia', 'Dima', 'Katherine', 'James', 'Emily',
             'Michael', 'Matthew', 'Laura', 'Kevin', 'Jonas'],
    'score': [12.5, 9, 16.5, np.nan, 9, 20, 14.5, np.nan, 8, 19],
    'attempts': [1, 3, 2, 3, 2, 3, 1, 1, 2, 1],
    'qualify': ['yes', 'no', 'yes', 'no', 'no',
                'yes', 'yes', 'no', 'no', 'yes']
}

labels = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j']

df = pd.DataFrame(exam_data, index=labels)

print("First 3 rows:")
print(df.head(3))

```

Output

```

>>>
===== RESTART: C:/Users/udhay/AppData/Local/Programs/Python/Python311/m.py =====
First 3 rows:
   name  score  attempts  qualify
a Anastasia  12.5         1     yes
b      Dima   9.0         3      no
c Katherine  16.5         2     yes

```

EXP 40

Aim

To write a Pandas program to select the name and score columns from the given DataFrame.

Algorithm

- Import Pandas and NumPy.
- Create the given DataFrame.
- Select the name column.
- Select the score column.
- Display both columns.

Code

```
import pandas as pd
import numpy as np

exam_data = {
    'name': ['Anastasia', 'Dima', 'Katherine', 'James', 'Emily',
            'Michael', 'Matthew', 'Laura', 'Kevin', 'Jonas'],
    'score': [12.5, 9, 16.5, np.nan, 9, 20, 14.5, np.nan, 8, 19],
    'attempts': [1, 3, 2, 3, 2, 3, 1, 1, 2, 1],
    'qualify': ['yes', 'no', 'yes', 'no', 'no',
               'yes', 'yes', 'no', 'no', 'yes']
}

labels = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j']
```

```
df = pd.DataFrame(exam_data, index=labels)
```

```
result = df[['name', 'score']]
```

```
print(result)
```

Output

```
>>> ===== RESTART: C:/Users/udhay/AppData/Local/Programs/Python/Python311/m.py =====
      name  score
a  Anastasia  12.5
b      Dima   9.0
c  Katherine  16.5
d      James  NaN
e      Emily   9.0
f   Michael  20.0
g   Matthew  14.5
h      Laura  NaN
i      Kevin   8.0
j      Jonas  19.0
>>> |
```